State Management in Cosmos SDK: Deep Dive

목차

- 개요
- KVStore 아키텍처
- MultiStore 시스템
- IAVL Tree 원리
- Context 객체
- State 접근 패턴
- Caching & Gas
- Pruning 전략
- Best Practices
- 참고 자료

개요

Cosmos SDK의 상태 관리는 블록체인의 현재 상태를 저장하고 조회하는 핵심 메커니즘입니다. 이는 **KVStore** (Key-Value Store)를 기반으로 하며, **IAVL Tree**를 통해 Merkle proof를 생성합니다.

학습 목표:

- KVStore의 계층 구조 완전 이해
- MultiStore와 CommitMultiStore의 역할
- IAVL tree의 작동 원리
- Context를 통한 상태 접근
- 효율적인 상태 관리 패턴

예상 학습 시간: 3-4시간

KVStore 아키텍처

Store 계층 구조

```
┌─────────────────────────────────────────┐
│             Application (BaseApp)       │
└─────────────────────────────────────────┘
                    │
                    ▼
┌─────────────────────────────────────────┐
│          RootMultiStore (CMS)           │
│  - 모든 모듈의 store를 관리              │
│  - Commit 시 모든 store를 한번에 커밋    │
└─────────────────────────────────────────┘
                    │
                    │
        ┌───────────┴───────────┐
        ▼           ▼           ▼
   ┌─────────┐  ┌─────────┐  ┌─────────┐
   │ Auth    │  │ Bank    │  │ Staking │
   │ Store   │  │ Store   │  │ Store   │
   │ (IAVL)  │  │ (IAVL)  │  │ (IAVL)  │
   └─────────┘  └─────────┘  └─────────┘
        │           │           │
        ▼           ▼           ▼
   [Accounts]  [Balances]  [Validators]
   [Sequences] [Supply]    [Delegations]
```

Store 타입

Cosmos SDK는 여러 종류의 store를 제공합니다:

1. CommitKVStore (IAVL Store)

용도: 영구 저장이 필요한 상태

```go
type CommitKVStore interface {
    KVStore
    Committer
}

type Committer interface {
    Commit() CommitID
    LastCommitID() CommitID
}
```

특징:

- Merkle tree 기반 (IAVL)
- 암호학적 증명 가능 (Merkle proof)
- 블록 높이별 버전 관리
- Disk에 영구 저장

사용 예:

```go
// 모듈의 메인 store
store := ctx.KVStore(k.storeKey)
store.Set(key, value)
```

2. TransientStore

용도: 블록 내에서만 유효한 임시 데이터

```
type TransientStore interface {
    KVStore
}
```

특징:

- Commit되지 않음
- 블록이 끝나면 삭제
- Merkle tree 없음 (증명 불가)
- 메모리에만 존재

사용 예:

```
// 블록 내 임시 카운터
tstore := ctx.TransientStore(k.tStoreKey)
tstore.Set([]byte("temp_counter"), sdk.Uint64ToBigEndian(count))
```

사용 사례:

- 블록 내 중복 체크
- 임시 집계 데이터
- Gas 계산용 임시 데이터

3. MemoryStore

용도: 인메모리 캐싱

```
type MemoryStore interface {
    KVStore
}
```

특징:

- 영구 저장되지 않음
- 노드 재시작 시 초기화
- 빠른 접근 속도

사용 예:

```
// 자주 조회되는 데이터 캐싱
memStore := ctx.KVStore(k.memStoreKey)
memStore.Set([]byte("cached_param"), value)
```

KVStore 인터페이스

```go
// store/types/store.go
type KVStore interface {
    // Get returns nil if key doesn't exist. Panics on error.
    Get(key []byte) []byte

    // Has checks if a key exists. Panics on error.
    Has(key []byte) bool

    // Set sets the key. Panics on error.
    Set(key, value []byte)

    // Delete deletes the key. Panics on error.
    Delete(key []byte)

    // Iterator over a domain of keys in ascending order.
    Iterator(start, end []byte) Iterator

    // ReverseIterator over a domain of keys in descending order.
    ReverseIterator(start, end []byte) Iterator
}
```

Store Key 설계

Store key는 데이터를 효율적으로 조직화하는 핵심입니다.

Key 구조 패턴

```go
// x/bank/types/keys.go

// Prefix를 사용한 key 구조
var (
    BalancesPrefix     = []byte{0x02}
    SupplyPrefix       = []byte{0x00}
    DenomMetadataPrefix = []byte{0x01}
)

// Balance key: prefix + address + denom
func BalanceKey(addr sdk.AccAddress, denom string) []byte {
    return append(append(BalancesPrefix, addr.Bytes()...), []byte(denom)...)
}

// 예: []byte{0x02} + cosmos1abc... + "stake"
// 결과: 0x02 + 14bytes_address + "stake"
```

Key 설계 Best Practices

```go
// ✅ Good: Hierarchical key structure
// Format: [prefix][entity_type][entity_id][sub_key]

const (
    PostPrefix     byte = 0x01
    CommentPrefix  byte = 0x02
    LikePrefix     byte = 0x03
)

// Post: 0x01 + uint64(postID)
func PostKey(id uint64) []byte {
    return append([]byte{PostPrefix}, sdk.Uint64ToBigEndian(id)...)
}

// Comment: 0x02 + uint64(postID) + uint64(commentID)
func CommentKey(postID, commentID uint64) []byte {
    key := []byte{CommentPrefix}
    key = append(key, sdk.Uint64ToBigEndian(postID)...)
    key = append(key, sdk.Uint64ToBigEndian(commentID)...)
    return key
}

// Like: 0x03 + uint64(postID) + address
func LikeKey(postID uint64, addr sdk.AccAddress) []byte {
    key := []byte{LikePrefix}
    key = append(key, sdk.Uint64ToBigEndian(postID)...)
    key = append(key, addr.Bytes()...)
    return key
}
```

장점:

- 효율적인 prefix iteration
- 명확한 데이터 구조
- 충돌 방지

MultiStore 시스템

CommitMultiStore

CommitMultiStore는 여러 개의 CommitKVStore를 관리합니다.

```go
// store/types/store.go
type CommitMultiStore interface {
    Committer
    MultiStore

    // Mount a store with a given key and DB
    MountStoreWithDB(key StoreKey, typ StoreType, db dbm.DB)

    // Load latest version
    LoadLatestVersion() error

    // Load a specific version
    LoadVersion(ver int64) error

    // Get commit info for a version
    GetCommitInfo(ver int64) *CommitInfo

    // Set pruning options
    SetPruning(pruning PruningOptions)

    // Get pruning options
    GetPruning() PruningOptions
}
```

Store 등록

```go
// app.go
func NewMyApp(...) *MyApp {
    // Store keys 정의
    keys := sdk.NewKVStoreKeys(
        authtypes.StoreKey,
        banktypes.StoreKey,
        stakingtypes.StoreKey,
        // ... 기타 모듈
    )

    tkeys := sdk.NewTransientStoreKeys(
        paramstypes.TStoreKey,
    )

    memKeys := sdk.NewMemoryStoreKeys(
        capabilitytypes.MemStoreKey,
    )

    // CommitMultiStore 생성
    app.cms = store.NewCommitMultiStore(db)

    // 각 store 마운트
    for _, key := range keys {
        app.cms.MountStoreWithDB(key, sdk.StoreTypeIAVL, nil)
    }
    for _, key := range tkeys {
        app.cms.MountStoreWithDB(key, sdk.StoreTypeTransient, nil)
    }
    for _, key := range memKeys {
        app.cms.MountStoreWithDB(key, sdk.StoreTypeMemory, nil)
    }

    // Store 로드
    if err := app.cms.LoadLatestVersion(); err != nil {
        panic(err)
    }

    return app
}
```

Commit 프로세스

```go
// baseapp/abci.go
func (app *BaseApp) Commit() abci.ResponseCommit {
    header := app.deliverState.ctx.BlockHeader()

    // 1. MultiStore에 모든 변경사항 커밋
    app.cms.Commit()

    // 2. App hash 계산
    commitID := app.cms.LastCommitID()

    // 3. State 정리
    app.deliverState = nil

    // 4. CheckState 리셋
    app.setCheckState(header)

    return abci.ResponseCommit{
        Data:         commitID.Hash,
        RetainHeight: app.GetRetainHeight(),
    }
}
```

Commit 시 발생하는 일:

1. **Write buffer flush**: 메모리의 변경사항을 IAVL tree에 반영
2. **Merkle root 계산**: 각 store의 root hash 계산
3. **App hash 생성**: 모든 store의 root hash를 조합
4. **Disk 저장**: IAVL tree를 database에 저장
5. **Version 증가**: 블록 높이 증가

```
Block Height N-1                    Block Height N
     │                                   │
     ▼                                   ▼
┌──────────┐                        ┌──────────┐
│MemCache  │  ── Commit() ──────>   │MemCache  │
└──────────┘                        └──────────┘
     │                                   │
     ▼                                   ▼
┌──────────┐                        ┌──────────┐
│IAVL Tree │                        │IAVL Tree │
│ Hash: A  │                        │ Hash: B  │
└──────────┘                        └──────────┘
     │                                   │
     ▼                                   ▼
┌──────────┐                        ┌──────────┐
│Database  │                        │Database  │
│Version N-1                        │Version N │
└──────────┘                        └──────────┘
```

IAVL Tree 원리

IAVL이란?

IAVL (Immutable AVL Tree): Self-balancing binary tree with Merkle tree properties

```
                  Root (Hash: R)
                 /              \
          Node A                 Node B
         (Hash: HA)             (Hash: HB)
         /        \             /        \
     Leaf 1     Leaf 2       Leaf 3     Leaf 4
    (key1)     (key2)       (key3)     (key4)
```

특징:

- **Self-balancing**: O(log n) 검색/삽입/삭제
- **Merkle tree**: 암호학적 증명 가능
- **Immutable**: 버전별 스냅샷 유지
- **Efficient**: 버전 간 차이만 저장 (Copy-on-Write)

AVL Balance

AVL tree는 각 노드의 좌우 서브트리 높이 차이가 최대 1이 되도록 유지합니다.

```go
// iavl/node.go
type Node struct {
    key       []byte
    value     []byte
    version   int64
    height    int8
    size      int64
    hash      []byte
    leftNode  *Node
    rightNode *Node
}

// Balance factor = height(right) - height(left)
// Must be in range [-1, 0, 1]
func (node *Node) calcBalance() int8 {
    return node.rightNode.height - node.leftNode.height
}
```

Rotation

Balance를 유지하기 위한 회전 연산:

Left Rotation

```
   A                B
  / \              / \
 X   B     ──>    A   Z
    / \          / \
   Y   Z        X   Y
```

Right Rotation

```
     B              A
    / \            / \
   A   Z    ──>   X   B
  / \                / \
 X   Y              Y   Z
```

Merkle Proof

IAVL tree는 특정 key-value 쌍의 존재를 증명할 수 있습니다.

```go
// Proof structure
type RangeProof struct {
    LeftPath   []PathToLeaf
    InnerNodes []PathToLeaf
    Leaves     []proofLeafNode
    RightPath  []PathToLeaf
}

// Verify proof
func (proof *RangeProof) Verify(rootHash []byte) error {
    // 1. Leaf hash 계산
    leafHash := proof.Leaves[0].Hash()

    // 2. Path를 따라 올라가며 hash 계산
    for _, pathNode := range proof.LeftPath {
        leafHash = computeHash(pathNode, leafHash)
    }

    // 3. 계산된 hash가 root hash와 일치하는지 확인
    if !bytes.Equal(leafHash, rootHash) {
        return errors.New("proof verification failed")
    }

    return nil
}
```

사용 사례:

- Light client verification
- IBC (Inter-Blockchain Communication)
- State sync

Versioning

IAVL은 버전별 스냅샷을 유지합니다.

```go
// Load a specific version
tree, err := iavl.NewMutableTree(db, cacheSize)
version, err := tree.LoadVersion(42)

// Get value at specific version
value, err := tree.GetVersioned(key, 42)

// Iterate over a version
tree.IterateVersioned(42, func(key, value []byte) bool {
    // Process key-value pair
    return false
})
```

Copy-on-Write 최적화:

변경되지 않은 노드는 재사용하여 저장 공간을 절약합니다.

```
Version 1:          Version 2: (only changed nodes)
    A                   A'
   / \                 /  \
  B   C      ——>      B'   C  (reused)
 /|   |\             / |
D E   F G           D' E (reused)
```

Context 객체

Context는 블록체인 상태와 메타데이터를 캡슐화합니다.

Context 구조

```go
// types/context.go
type Context struct {
    ctx           context.Context
    ms            MultiStore
    header        cmtproto.Header
    chainID       string
    txBytes       []byte
    logger        log.Logger
    voteInfo      []abci.VoteInfo
    gasMeter      GasMeter
    blockGasMeter GasMeter
    checkTx       bool
    minGasPrice   DecCoins
    consParams    *cmtproto.ConsensusParams
    eventManager  *EventManager
}
```

Context 생성

```go
// baseapp/baseapp.go
func (app *BaseApp) getContextForTx(mode runTxMode, txBytes []byte) sdk.Context {
    var ctx sdk.Context

    switch mode {
    case runTxModeCheck:
        ctx = app.checkState.ctx
    case runTxModeDeliver, runTxModeSimulate:
        ctx = app.deliverState.ctx
    default:
        panic(fmt.Sprintf("invalid runTxMode: %d", mode))
    }

    ctx = ctx.
        WithTxBytes(txBytes).
        WithVoteInfos(app.voteInfos).
        WithConsensusParams(app.GetConsensusParams(ctx))

    if mode == runTxModeSimulate {
        ctx, _ = ctx.CacheContext()
    }

    return ctx
}
```

Context Chaining

Context는 불변(immutable)이며, 변경 시 새로운 context를 반환합니다.

```
// ✅ Good: Context chaining
ctx = ctx.
    WithBlockHeight(100).
    WithBlockTime(time.Now()).
    WithChainID("mychain-1").
    WithGasMeter(sdk.NewGasMeter(1000000))

// ❌ Bad: Trying to mutate context
ctx.BlockHeight = 100  // Compile error!
```

Context 메소드

블록 정보 접근

```
// Block metadata
blockHeight := ctx.BlockHeight()      // int64
blockTime := ctx.BlockTime()          // time.Time
chainID := ctx.ChainID()              // string
header := ctx.BlockHeader()           // cmtproto.Header

// Transaction metadata
txBytes := ctx.TxBytes()              // []byte
isCheckTx := ctx.IsCheckTx()          // bool
```

Store 접근

```
// Get KVStore
store := ctx.KVStore(storeKey)

// Get TransientStore
tstore := ctx.TransientStore(tstoreKey)
```

Gas 관리

```
// Get gas meter
gasMeter := ctx.GasMeter()

// Consume gas
ctx.GasMeter().ConsumeGas(10000, "my operation")

// Check gas
if ctx.GasMeter().IsOutOfGas() {
    panic("out of gas")
}

// Get gas consumed
gasUsed := ctx.GasMeter().GasConsumed()
```

이벤트 관리

```
// Emit event
ctx.EventManager().EmitEvent(
    sdk.NewEvent(
        "my_event",
        sdk.NewAttribute("key", "value"),
    ),
)

// Get all events
events := ctx.EventManager().Events()
```

Cache Context

Cache context는 상태 변경을 격리하고, 선택적으로 적용할 수 있습니다.

```
// Create cache context
cacheCtx, writeFn := ctx.CacheContext()

// Make changes to cache context
store := cacheCtx.KVStore(storeKey)
store.Set(key, value)

// Option 1: Apply changes
writeFn()  // Changes are written to parent context

// Option 2: Discard changes
// Simply don't call writeFn(), changes are lost
```

사용 사례:

```
// Try operation, rollback on error
func (k Keeper) TryOperation(ctx sdk.Context) error {
    cacheCtx, writeFn := ctx.CacheContext()

    if err := k.riskyOperation(cacheCtx); err != nil {
        // Error occurred, don't call writeFn()
        // Changes to cacheCtx are discarded
        return err
    }

    // Success, apply changes
    writeFn()
    return nil
}
```

State 접근 패턴

기본 CRUD 패턴

```go
type Keeper struct {
    storeKey sdk.StoreKey
    cdc      codec.BinaryCodec
}

// Create/Update
func (k Keeper) SetUser(ctx sdk.Context, user types.User) {
    store := ctx.KVStore(k.storeKey)
    key := types.UserKey(user.Address)
    bz := k.cdc.MustMarshal(&user)
    store.Set(key, bz)
}

// Read
func (k Keeper) GetUser(ctx sdk.Context, addr string) (types.User, bool) {
    store := ctx.KVStore(k.storeKey)
    key := types.UserKey(addr)
    bz := store.Get(key)
    if bz == nil {
        return types.User{}, false
    }

    var user types.User
    k.cdc.MustUnmarshal(bz, &user)
    return user, true
}

// Delete
func (k Keeper) DeleteUser(ctx sdk.Context, addr string) {
    store := ctx.KVStore(k.storeKey)
    key := types.UserKey(addr)
    store.Delete(key)
}
```

Iterator 패턴

```go
// Iterate all users
func (k Keeper) IterateUsers(ctx sdk.Context, cb func(user types.User) bool) {
    store := ctx.KVStore(k.storeKey)
    iterator := sdk.KVStorePrefixIterator(store, types.UserPrefix)
    defer iterator.Close()

    for ; iterator.Valid(); iterator.Next() {
        var user types.User
        k.cdc.MustUnmarshal(iterator.Value(), &user)

        if cb(user) {
            break
        }
    }
}

// Get all users
func (k Keeper) GetAllUsers(ctx sdk.Context) []types.User {
    users := []types.User{}
    k.IterateUsers(ctx, func(user types.User) bool {
        users = append(users, user)
        return false
    })
    return users
}

// Get users with pagination
func (k Keeper) GetUsersPaginated(
    ctx sdk.Context,
    pageReq *query.PageRequest,
) ([]types.User, *query.PageResponse, error) {
    store := ctx.KVStore(k.storeKey)
    userStore := prefix.NewStore(store, types.UserPrefix)

    users := []types.User{}
    pageRes, err := query.Paginate(userStore, pageReq, func(key []byte, value []byte) error {
        var user types.User
        if err := k.cdc.Unmarshal(value, &user); err != nil {
            return err
        }
        users = append(users, user)
        return nil
    })

    return users, pageRes, err
}
```

Index 패턴

Primary key 외에 secondary index를 관리합니다.

```go
// Primary key: UserPrefix + address
func UserKey(addr string) []byte {
    return append(UserPrefix, []byte(addr)...)
}

// Secondary index: UserByEmailPrefix + email -> address
func UserByEmailKey(email string) []byte {
    return append(UserByEmailPrefix, []byte(email)...)
}

// Set user with index
func (k Keeper) SetUser(ctx sdk.Context, user types.User) {
    store := ctx.KVStore(k.storeKey)

    // 1. Set primary data
    key := UserKey(user.Address)
    bz := k.cdc.MustMarshal(&user)
    store.Set(key, bz)

    // 2. Set secondary index
    indexKey := UserByEmailKey(user.Email)
    store.Set(indexKey, []byte(user.Address))
}

// Get user by email
func (k Keeper) GetUserByEmail(ctx sdk.Context, email string) (types.User, bool) {
    store := ctx.KVStore(k.storeKey)

    // 1. Get address from index
    indexKey := UserByEmailKey(email)
    addr := store.Get(indexKey)
    if addr == nil {
        return types.User{}, false
    }

    // 2. Get user by address
    return k.GetUser(ctx, string(addr))
}

// Delete user (clean up index)
func (k Keeper) DeleteUser(ctx sdk.Context, addr string) {
    store := ctx.KVStore(k.storeKey)

    // 1. Get user to find email
    user, found := k.GetUser(ctx, addr)
    if !found {
        return
    }

    // 2. Delete secondary index
    indexKey := UserByEmailKey(user.Email)
    store.Delete(indexKey)

    // 3. Delete primary data
    key := UserKey(addr)
    store.Delete(key)
}
```

Caching & Gas

CacheKVStore

CacheKVStore는 읽기/쓰기를 캐싱하여 성능을 향상시킵니다.

```go
// store/cachekv/store.go
type Store struct {
    parent    types.KVStore
    cache     map[string]*cValue
    deleted   map[string]struct{}
    sortedKey []string
}

// Read operations check cache first
func (s *Store) Get(key []byte) []byte {
    types.AssertValidKey(key)

    cacheKey := conv.UnsafeBytesToStr(key)

    // Check if deleted
    if _, ok := s.deleted[cacheKey]; ok {
        return nil
    }

    // Check cache
    if cValue, ok := s.cache[cacheKey]; ok {
        return cValue.value
    }

    // Read from parent
    value := s.parent.Get(key)
    s.setCacheValue(cacheKey, value, false, false)

    return value
}

// Write operations write to cache
func (s *Store) Set(key, value []byte) {
    types.AssertValidKey(key)
    types.AssertValidValue(value)

    cacheKey := conv.UnsafeBytesToStr(key)
    s.setCacheValue(cacheKey, value, false, true)
}

// Write cache to parent on commit
func (s *Store) Write() {
    for _, key := range s.sortedKey {
        if cValue, ok := s.cache[key]; ok {
            if cValue.dirty {
                if cValue.delete {
                    s.parent.Delete([]byte(key))
                } else {
                    s.parent.Set([]byte(key), cValue.value)
                }
            }
        }
    }

    // Clear cache
    s.cache = make(map[string]*cValue)
    s.deleted = make(map[string]struct{})
    s.sortedKey = nil
}
```

Gas Metering

모든 store 접근은 gas를 소비합니다.

```go
// store/gaskv/store.go
type Store struct {
    parent    types.KVStore
    gasMeter  types.GasMeter
    gasConfig types.GasConfig
}

func (s *Store) Get(key []byte) []byte {
    s.gasMeter.ConsumeGas(s.gasConfig.ReadCostFlat, types.GasReadCostFlatDesc)

    value := s.parent.Get(key)

    // Charge gas per byte
    s.gasMeter.ConsumeGas(
        s.gasConfig.ReadCostPerByte*types.Gas(len(key)),
        types.GasReadPerByteDesc,
    )
    s.gasMeter.ConsumeGas(
        s.gasConfig.ReadCostPerByte*types.Gas(len(value)),
        types.GasReadPerByteDesc,
    )

    return value
}

func (s *Store) Set(key, value []byte) {
    s.gasMeter.ConsumeGas(s.gasConfig.WriteCostFlat, types.GasWriteCostFlatDesc)
    s.gasMeter.ConsumeGas(
        s.gasConfig.WriteCostPerByte*types.Gas(len(key)),
        types.GasWritePerByteDesc,
    )
    s.gasMeter.ConsumeGas(
        s.gasConfig.WriteCostPerByte*types.Gas(len(value)),
        types.GasWritePerByteDesc,
    )

    s.parent.Set(key, value)
}
```

기본 Gas 비용:

```go
// store/types/gas.go
var (
    DefaultGasConfig = GasConfig{
        ReadCostFlat:     1000,
        ReadCostPerByte:  3,
        WriteCostFlat:    2000,
        WriteCostPerByte: 30,
        IterNextCostFlat: 30,
    }
)
```

Pruning 전략

Pruning은 오래된 버전을 삭제하여 디스크 공간을 절약합니다.

Pruning 옵션

```go
// store/types/pruning.go
type PruningOptions struct {
    // KeepRecent: 최근 N개 버전 유지
    KeepRecent uint64

    // Interval: 매 N개 블록마다 pruning 실행
    Interval uint64
}

// Predefined strategies
var (
    // PruneNothing: 모든 버전 유지 (archive node)
    PruneNothing = NewPruningOptions(0, 0)

    // PruneDefault: 최근 362,880개 블록 유지 (약 3주)
    PruneDefault = NewPruningOptions(362880, 10)

    // PruneEverything: 최소한만 유지 (최신 상태만)
    PruneEverything = NewPruningOptions(2, 10)
)
```

Pruning 설정

```toml
# app.toml
[pruning]
# Pruning strategy: default, nothing, everything, custom
pruning = "default"

# 커스텀 pruning 옵션
pruning-keep-recent = "362880"
pruning-interval = "10"
```

```go
// app.go
func (app *MyApp) SetPruning(pruningOpts sdk.PruningOptions) {
    app.cms.SetPruning(pruningOpts)
}
```

Pruning 실행

```go
// store/rootmulti/store.go
func (rs *Store) Commit() types.CommitID {
    // 1. Commit all stores
    for key, store := range rs.stores {
        commitID := store.Commit()
        // ...
    }

    // 2. Increment version
    rs.lastCommitInfo = rs.flushCommitInfo()

    // 3. Prune if needed
    if rs.pruningManager.ShouldPrune(rs.lastCommitInfo.Version) {
        rs.pruningManager.Prune(rs.lastCommitInfo.Version)
    }

    return commitID
}

// Pruning logic
func (pm *PruningManager) Prune(currentHeight int64) error {
    pruneHeight := currentHeight - int64(pm.opts.KeepRecent)

    if pruneHeight <= 0 {
        return nil
    }

    // Delete old versions from IAVL
    for _, store := range pm.stores {
        if err := store.DeleteVersion(pruneHeight); err != nil {
            return err
        }
    }

    return nil
}
```

Best Practices

1. Key Design

```go
// ✅ Good: Use prefixes and fixed-size encoding
const (
    BalancePrefix     = 0x01
    DelegationPrefix  = 0x02
)

func BalanceKey(addr sdk.AccAddress, denom string) []byte {
    // prefix + address + denom
    key := []byte{BalancePrefix}
    key = append(key, addr.Bytes()...)
    key = append(key, []byte(denom)...)
    return key
}

// ❌ Bad: Variable encoding or string concatenation
func BadBalanceKey(addr, denom string) []byte {
    return []byte(addr + ":" + denom)  // No prefix, hard to iterate
}
```

2. Efficient Iteration

```go
// ✅ Good: Use prefix iterator
func (k Keeper) IterateBalances(ctx sdk.Context, addr sdk.AccAddress, cb func(sdk.Coin) bool) {
    store := ctx.KVStore(k.storeKey)
    prefix := append([]byte{BalancePrefix}, addr.Bytes()...)
    iterator := sdk.KVStorePrefixIterator(store, prefix)
    defer iterator.Close()

    for ; iterator.Valid(); iterator.Next() {
        // Process
    }
}

// ❌ Bad: Iterate all and filter
func (k Keeper) BadIterateBalances(ctx sdk.Context, addr sdk.AccAddress, cb func(sdk.Coin) bool) {
    store := ctx.KVStore(k.storeKey)
    iterator := sdk.KVStorePrefixIterator(store, []byte{BalancePrefix})
    defer iterator.Close()

    for ; iterator.Valid(); iterator.Next() {
        // Parse key, check address - inefficient!
    }
}
```

3. Use Cache Context for Validation

```go
// ✅ Good: Use cache context for dry-run
func (k Keeper) ValidateOperation(ctx sdk.Context) error {
    cacheCtx, _ := ctx.CacheContext()

    if err := k.tryOperation(cacheCtx); err != nil {
        return err
    }

    // Don't call writeFn, changes are discarded
    return nil
}
```

4. Gas Awareness

```go
// ✅ Good: Consume gas explicitly for expensive operations
func (k Keeper) ExpensiveComputation(ctx sdk.Context, data []byte) {
    // Charge gas based on input size
    gasAmount := uint64(len(data)) * GasPerByte
    ctx.GasMeter().ConsumeGas(gasAmount, "expensive computation")

    // Do computation
}
```

5. Index Management

```go
// ✅ Good: Clean up indexes when deleting
func (k Keeper) DeletePost(ctx sdk.Context, id uint64) {
    store := ctx.KVStore(k.storeKey)

    // Get post to find indexes
    post, _ := k.GetPost(ctx, id)

    // Delete indexes
    store.Delete(PostByCreatorKey(post.Creator, id))
    store.Delete(PostByTimestampKey(post.Timestamp, id))

    // Delete main entry
    store.Delete(PostKey(id))
}
```

참고 자료

공식 문서

- Cosmos SDK Store Package
- IAVL Specification
- Context Documentation

소스 코드

- Store Types
- IAVL Implementation
- Context Implementation

추가 자료

- ADR-040: Storage and SMT State Commitments
- IAVL Paper

다음 단계

State management를 이해했다면, 다음 주제로 넘어갈 수 있습니다:

1. **Transaction Lifecycle** → 05-transaction-lifecycle.md
    - 트랜잭션이 상태를 어떻게 변경하는지
    - Gas 계산 메커니즘
    - Transaction lifecycle tracer 개발
2. **Cosmos EVM** → 06-cosmos-evm-architecture.md
    - EVM StateDB와 Cosmos SDK Store 통합
    - Storage slot 관리

작성자: B-Harvest Core Developer Candidate **버전**: 1.0 **최종 수정일**: 2025-01-XX